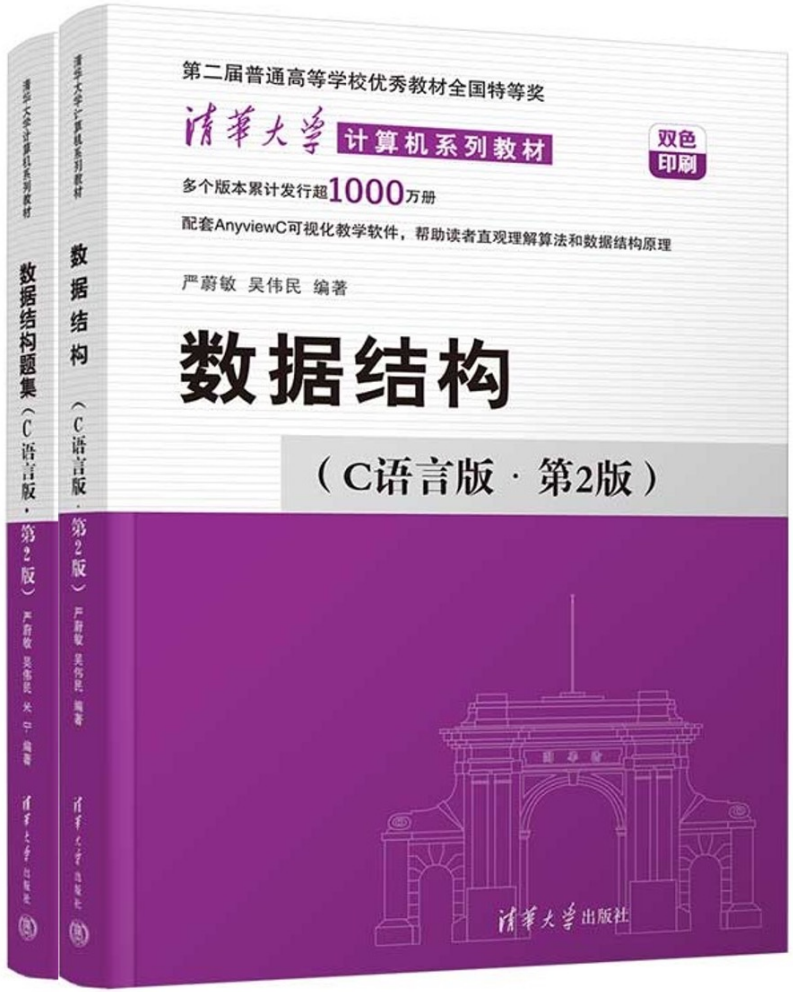


清華大学出版社
TSINGHUA UNIVERSITY PRESS

第4章

串

4.1 串类型的定义

串（string)（或字符串）是由零个或多个字符组成的有限序列，一般记为

$$s = "a_1a_2\cdots a_n" \quad (n \geqslant 0) \qquad (4\text{-}1)$$

其中，s是串的名，用双引号括起来的字符序列是串的值；a_i $(1 \leqslant i \leqslant n)$ 可以是字母、数字或其他字符。串中字符的数目n称为串的**长度**。零个字符的串称为**空串**（null string)，它的长度为零。

串中任意个连续的字符组成的子序列称为该串的**子串**。包含子串的串相应地称为**主串**。通常称字符在序列中的序号为该字符在串中的**位置**。子串在主串中的位置则以子串的第一个字符在主串中的位置来表示。

例如，设a、b、c、d为如下的4个串：

a = "BEI"　　b = "JING"

c = "BEIJING"　　d = "BEI JING"

它们的长度分别为3、4、7和8；并且a和b都是c和d的子串，a在c和d中的位置都是1，而b在c中的位置是4，在d中的位置则是5，d中第4个字符是空格。

称两个串是**相等**的，当且仅当这两个串的值相等。即，只有当两个串的长度相等，且各个对应位置的字符都相等时才相等。例如上例中的串a、b、c和d彼此都不相等。

串值必须用一对双引号（与流行的程序设计语言一致，过去也有约定是单引号的）括起来，但双引号本身不属于串，它的作用是避免与变量名或数的常量混淆而已。

例如在程序设计语言中

```
x = "123";
```

表明x是串变量名，赋给它的值是字符序列123。又如

```
tsing = "TSING"
```

中，tsing是一个串变量名，而字符序列TSING是其值。

在各种应用中，**空格**常是串的字符集合中的一个元素，可出现在其他字符之间。由一个或多个空格组成的串"　　"称为**空格串**（blank string，请注意：这不是空串），它的长度为串中空格字符的个数。为了清楚起见，需要时可用符号"∅"来表示“空串”。

串的抽象数据类型

清華大學出版社 TSINGHUA UNIVERSITY PRESS

ADT String {

数据对象：**D** = { a_i | a_i∈CharacterSet，i = 1,2,…,n， n≥0 }

数据关系：**R1** = {<a_{i-1},a_i> | a_{i-1},a_i∈**D**，i = 2,…,n }

基本操作：

StrNew(cs);

初始条件：cs是C语言的字符串（可简称C串）。

操作结果：构造并返回一个值由C串cs复制的串。

StrFree(S);

初始条件：串S存在。

操作结果：回收串S。

StrClear(S);

初始条件：串S存在。

操作结果：将S清为空串。

StrEmpty(S);

初始条件：串S存在。

操作结果：若S为空串，则返回TRUE，否则返回FALSE。

StrLen(S);

初始条件：串S存在。

操作结果：返回S的元素个数，称为串的长度。

StrCmp(S, T);

初始条件：串S和T存在。

操作结果：若S>T，则返回值＞0；若S=T，则返回值＝0；
若S<T，则返回值＜0。

StrCopy(S);

初始条件：串S存在。

操作结果：由串S复制并返回新串。

StrConcat(S1, S2);

初始条件：串S1和S2存在。

操作结果：返回由S1和S2联接而成的新串。

StrSub(S, pos, len);

初始条件：串S存在，1≤pos≤StrLen(S)且
0<len≤StrLen(S)-pos+1。

操作结果：返回串S的第pos个字符起长度为len的子串。

SStrRange(S, a, b);

初始条件：串S存在，1≤a≤b≤StrLen(S)。

操作结果：保留S在区间[a..b]内的串值，其余被覆盖或清除。

Index(S, T, pos);

初始条件：串S和T存在，T是非空串，1≤pos≤StrLen(S)。

操作结果：若主串S中存在和串T值相同的子串，则返回它在主串S中第pos个字符起第一次出现的位置；否则函数值为0。

StrAssign(S, pos, cs);

初始条件：cs是C字符串。

操作结果：用C串cs复盖S中第pos个字符起的串值。

StrInsert(S, pos, cs);

初始条件：串S和T存在，1≤pos≤StrLen(S)+1，cs是C串。

操作结果：在串S的第pos个字符之前插入C串cs。

StrDelete(S, pos, len);

初始条件：串S存在，1≤pos≤StrLen(S)-len+1。

操作结果：从串S中删除第pos个字符起长度为len的子串。

} ADT String

4.2 串的表示和实现

4.2.1 C语言串的存储表示

C语言的串常量也叫**串字面值**（string literal）是用一对双引号"括起来的0或多个字符组成的字符序列。例如

```
"Data Structures"
```

或 `""` `// 空串`

都是串。双引号不是字符串的一部分，只用于限定串值。

从存储表示看，串常量就存储在一维字符数组。串的内部表示不存储那对双引号，但在末尾附加一个空字符'\0'（null）作为**结束符**。因此存储字符串的物理存储单元数比双引号内的字符数多一个①。这表明C语言未限制串的长度，但程序必须扫描完整的串值后才能确定串的长度。

C语言没有给串类型命名。可如下几种形式声明串变量

```
char str1[15];
     // 分配16个字符单元，最多容纳15个字符（末尾留一个单元存'\0'）
char str2[] = "Data Structures";
     // 在栈区分配16个字符单元，并将串值存入数组，末尾加'\0'
char *str3 = "Data Structures";
     // 在堆区分配16个字符单元，并将串值存入数组，末尾加'\0'
char *str4;          // 字符类型指针，未分配串值存储空间
```

其中，str2和str3分别声明为字符数组和字符指针。str2将始终指向在栈区为它分配的存储区域的基址，而str3当前指向了为给定串值分配的存储区域，之后可以修改以指向其他地址，但原所指向的区域可能要给与释放，防止内存泄漏。这也正是C语言的串的内存安全问题之一。图4.1示意了这两个变量的存储表示。

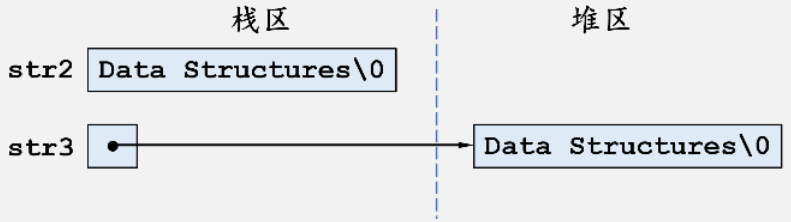


图4.1 C语言的字符数组和字符指针变量的存储表示示例

4.2.2 短串的存储表示

清華大學出版社 TSINGHUA UNIVERSITY PRESS

类似于线性表的顺序存储结构，可用一组地址连续的存储单元存储串值的字符序列。各种程序中频繁使用的多数是长度很短的串，长度被限制在254以内的串称为**短串**（short string），其存储表示描述如下。

```
//----- 短串的顺序存储表示及存储分配和销毁操作的实现 -----
#define  MAXSTRLEN   254    // 串长必须在254以内
typedef  char  *SStr;       // 动态分配串长+2个字符单元，0号单元存串长，串值末附加结束符'\0'
SStr allocSStr(int len) {   // 辅助操作：申请为S分配len+2容量的空间（可能被限长）
   SStr S;
   if (len > MAXSTRLEN) len = MAXSTRLEN;          // 最长不超过MAXSTRLEN
   if (!(S = (SStr)calloc(len+2, sizeof(char)))) exit(OVERFLOW); // 分配空间
   S[0] = len;                  // 借S[0]返回实际分配的容量，未赋串值前还不是真实串长
   return OK;
}
SStr SStrFree(SStr S) { free(S);   return NULL; }   // 回收操作
```

新建一个短串，先调用辅助操作allocSStr申请分配存储空间，其存储容量可在256（是串的长度+2）以内指定；然后存入串值，若串值有超过预定长度的部分则被舍去，称之为“截断”。

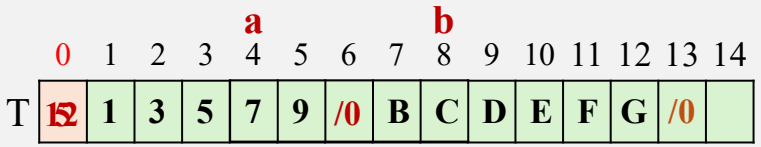
算法 4.3　求子串 SStrSub(S,pos,len)




```
SStr  ss = SStrSub(T, 4, 5);
```

```
SStr SStrSub (SStr S, int pos, int len) {
  // 返回串S的第pos个字符起长度为len的子串
  // 其中，1≤pos≤StrLen(S) 且 0≤len≤StrLen(S)-pos+1
  if (!S || pos<1 || pos>S[0] || len<0 || len>S[0]-pos+1) // 检查参数
    return NULL;
  SStr Sub = allocSStr(len);                  // 分配结果子串Sub的空间
  memcpy(Sub+1, S+pos, len);                  // 复制子串值
  Sub[0] = len;   Sub[len+1] = '\0';          // 子串长度，结束符
  return Sub;
}
```

算法 4.4 留子串 SStrRange(S, a, b)

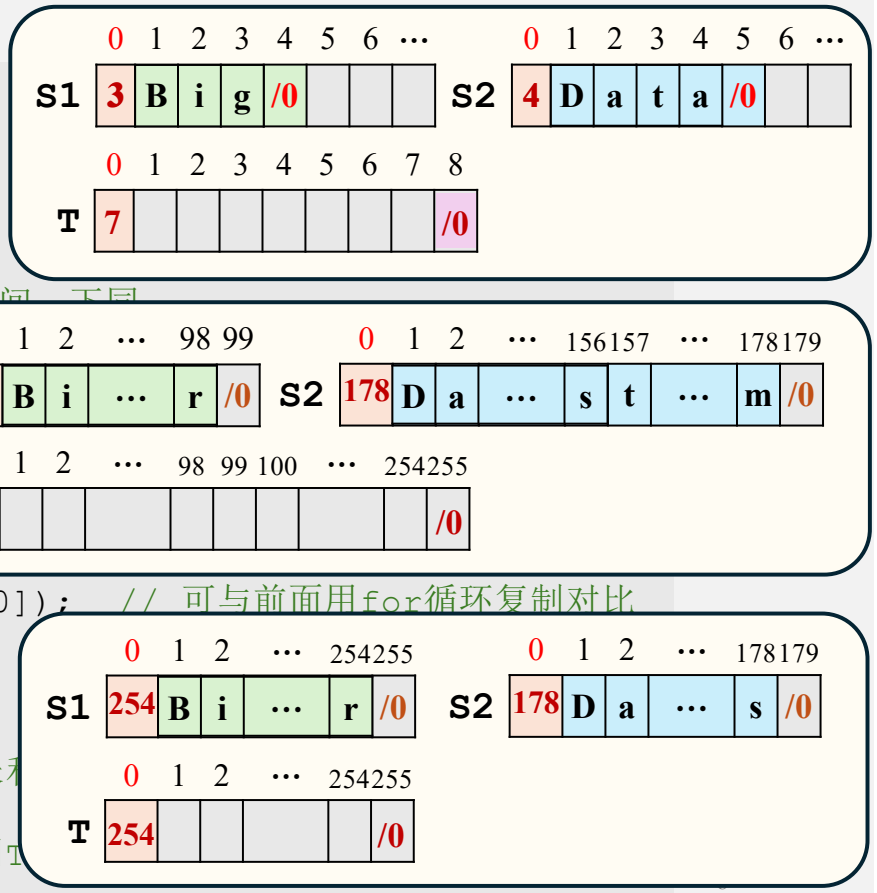


SStrRange(T, 4, 8);

```
Status SStrRange(SStr S, int a, int b) {
   // 保留S在区间[a-1..b-1]内的串值，其余被覆盖或清除。O(N),N为被保留空间容量
   if (!S || a<1 || a>b || b>S[0]) return ERROR;  // 合法性检查
   S[0] = b-a+1;                          // 更新串长
   memcpy(S+1, S+a, (int)S[0]);           // 要保留的串值移至起始位置
   S[1+S[0]] = '\0';                      // 结束符
   return OK;
}
```

算法 4.5　串连接 SStrConcat(S1,S2)

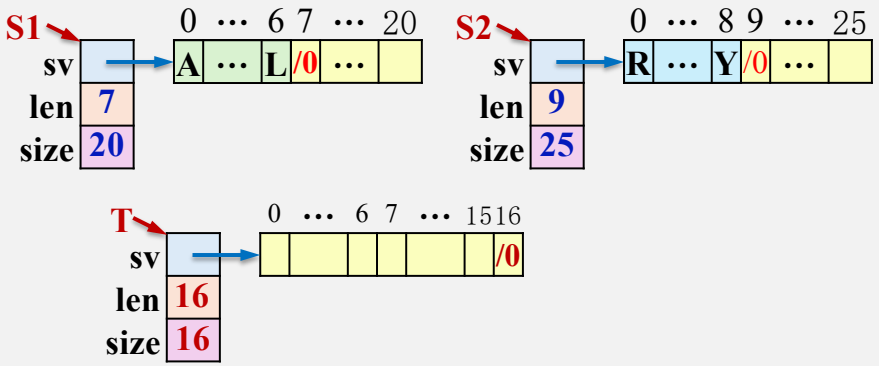
```
SStr SStrConcat(SStr S1, SStr S2) { // 连接操作
  // 用T返回由S1和S2连接而成的新串
  SStr T;    int i;
  if (!S1 || !S2) return  ERROR;     // 合法性检查
  if (S1[0]+S2[0] <= MAXSTRLEN) {    // 无需截断
    T = allocSStr(S1[0]+S2[0]);      // 按需分配空间，不同
①   for (i=1; i<=S1[0]; i++) T[i] = S1[i];
    for (i=1; i<=S2[0]; i++) T[i+S1[0]] =
    T[0] = S1[0]+S2[0];
 } else if (S1[0] < MAXSTRLEN) {   // 截断
    T = allocSStr(MAXSTRLEN);
②   memcpy(T+1, S1+1, (int)S1[0]);
    memcpy(T+(int)S1[0]+1, S2+1, MAXSTRLEN-S1[0]);   // 可与前面用for循环复制对比
    T[0] = MAXSTRLEN;
  }else {                            // 截断(仅取S1)
③   T = allocSStr(MAXSTRLEN);
    memcpy(T, S1, S1[0]);            // i从0起，包括串长
  }
  T[T[0]+1] = '\0';    return T; // 置结束符，返回T
}
```



算法 4.6　长串连接 LStrConcat(S1,S2)

清華大學出版社 TSINGHUA UNIVERSITY PRESS

```
//----- 长串的存储表示 -----
typedef struct {
  char *sv;     // 串值数组。容量为size+1
  int   len;    // 串的长度，串值的字符个数
  int   size;   // 容量，size-len是未用容量
} *LStr;        // 长串指针类型
```




```
LStr LStrConcat(LStr S1, LStr S2) {  // 构造并返回由S1和S2联接所得的新串
  LStr T;   int len;
  if (!S1 || !S2) return NULL;  // S1或S2不存在，则返回NULL保错
  if (!(T = (LStr)malloc(sizeof(*T)))) exit(OVERFLOW);     // 分配新串结构记录空间
  if (0==(len = S1->len+S2->len)) { T->sv=NULL;  T->len=0;  return S; } // 空串
  if (!(T->sv = (char*)malloc((len+1)*sizeof(char)))) exit(OVERFLOW);  // 串值空间
  memcpy(T->sv, S1->sv, S1->len);                  // 复制S1串值
  memcpy(T->sv+S1->len, S2->sv, S2->len);          // 复制S2串值
  T->len = T->size = len;   T->sv[len] = '\0'; // 置串T长度、容量和结束符
  return T;   // 返回联接构造的串T
}
```

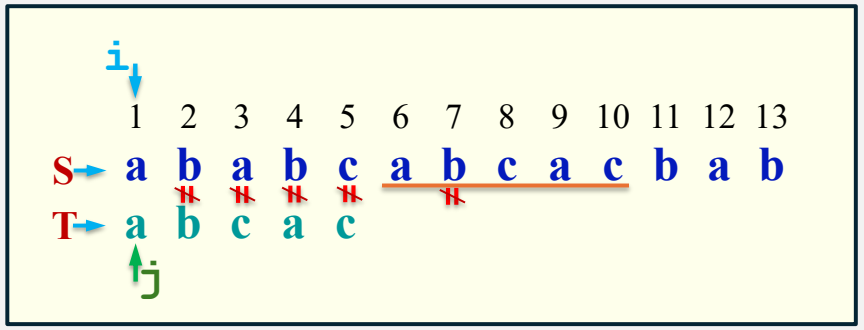
4.3.1 求子串位置的定位函数 Index(S,T,pos)

子串的定位操作通常称做串的**模式匹配**（其中T称为模式串），是各种串处理系统中最重要的操作之一。

4.1节的**算法4.1**是调用串的基本操作给出的定位函数。

根据算法4.1的基本思想，**算法4.8**采用短串存储结构，是不依赖于其他串操作的匹配算法。

图4.4 示例



1

```
int Index(SStr S, SStr T, int pos) {  // 算法 4.8  子串定位
  // 返回子串T在主串S中第pos个字符起的位置。若不存在，则函数值为0
  // 其中，T非空，1≤pos≤StrLen(S)
  int i=pos, j=1;
  if (!S || !T || T[0]<=0 || pos<1 || pos>S[0]) return 0; // 检查参数合法性
  while (i <= S[0] && j <= T[0])          // i和j分别指示S和T当前比较的字符位置
    if (S[i] == T[j]) { ++i;    ++j; }    // 继续比较后续字符
    else { i = i-j+2;    j = 1; }         // 指针回退，重新开始匹配
  if (j > T[0]) return  i-T[0];           // 成功，返回匹配子串位置
  else return 0;                          // 失败，返回0
}
```

4.3.2 模式匹配的一种改进算法 —— KMP算法

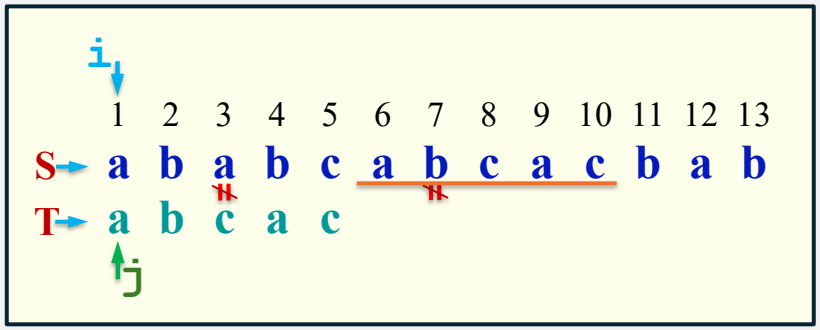
对**算法4.8**的改进：

（1）增设模式串的next函数值数组；

（2）修改**while**循环内的**else**子句。

图4.5的示例

j	1	2	3	4	5	6	7	8
模式串	a	b	a	a	b	c	a	c
next[j]	0	1	1	2	2	3	1	2




```
int Index_KMP(SStr S, SStr T, int pos) {   // 算法 4.9
  // 利用模式串T的next表求T在主串S中第pos个字符之后的位置的KMP算法
  // 其中，T非空，1≤pos≤StrLen(S)
  if (!S || !T || T[0]<=0 || pos<1 || pos>S[0]) return 0;   // 检查参数合法性
  int i=pos, j=1;
  while (i<=S[0] && j<=T[0])                          // i和j分别指示S和T当前比较的字符位置
    if (j==0 || S[i]==T[j]) { ++i;    ++j; }         // 继续比较后续字符
    else j = next[j];                                 // 模式串向右移动
  if (j > T[0]) return  i-T[0];                       // 匹配成功
  else return 0;
}
```

求模式串的next数组

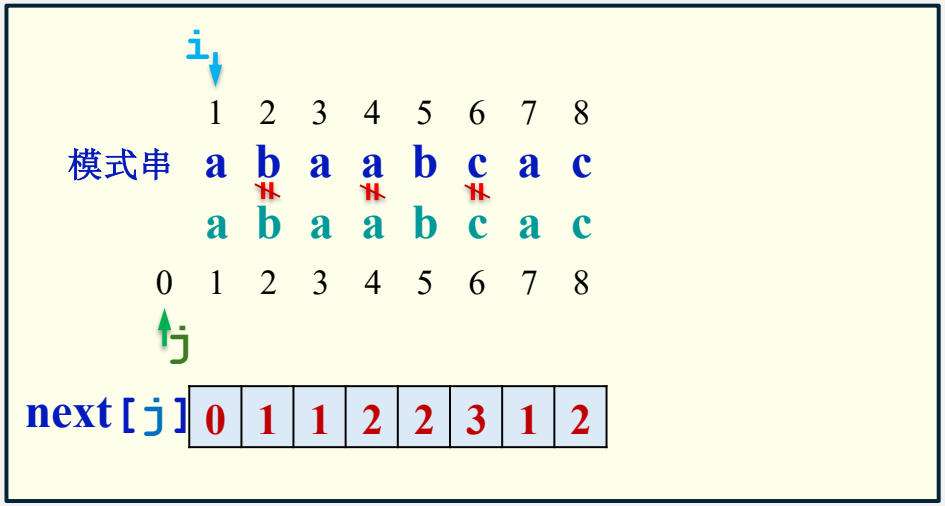
由**算法4.9**可衍生求模式串next数组值的**算法4.10**：

（1）初始置next[1]为 **0** ；

（2）递推next[i]的值，i=2,3,…, n，n为模式串长度。

图4.7的示例，求以下模式串的next数组值的过程

j	1	2	3	4	5	6	7	8
模式串	a	b	a	a	b	c	a	c
next[j]	0	1	1	2	2	3	1	2




```
void get_next(SStr T, int *next) {   // 算法 4.10 求模式串T的next函数值并存入数组next
  int i = 1, j = 0;    next[1] = 0;
  while (i < T[0])                    // 用i扫描T
    if (j == 0 || T[i] == T[j])       // j==0是初次
      { ++i;    ++j;    next[i] = j; } // 求得next[i]
    else j = next[j];                 // 向右滑动
}
```

4.4.2 建立词索引表

建议在AnyviewC可视交互跟踪算法过程

```
WordList  wdlist;   // 词表 可视跟踪设置：线性表
IdxList   idxlist;    // 索引表 可视跟踪设置：索引表
int main() {  // 算法4.12
char *bn_titles[] = { "005 Computer Data Structures",
                      "010 Introduction to Data Structures",
                      "023 Fundamentals of Data Structures",
                      "034 The Design and Analysis of Computer Algorithms",
                      "050 Introduction to Numerical Analysis",
                      "067 Numerical Analysis",
                      ""  };
  int BookNo;
  printf(" ---- 处理书目数据，构造数目索引表 ----\n");
  idxlist = InitIdxList();       // 初始化索引表idxl为空表
  wdlist  = InitWordList();
  for (int i=0; bn_titles[i]!= “ ” ; i++) {  // 依次对每一本书建索引
    printf(" %s\n", bn_titles[i]);
    BookNo = ExtractKeyWord(bn_titles[i], wdlist);  // 从书名提取关键词到词
    InsertIdxList(idxlist, wdlist, BookNo);    // 将书号为BookNo的关键词插入
  }
  printf(" ---------- 索引表 ----------\n");
  PrintIdxList(idxlist);
  return 0;
}
```

初始化

输出请见教材图4.11

